

Travaux pratiques 4 –RNN & LSTM

Exercice 1 : Données séquentielles

Comprendre la structure des données séquentielles.

Avec tensorflow créer une séquence de données et analyser sa forme.

Exercice 2: RNN Simple

Découvrir le fonctionnement d'un RNN.

1. **Importer les bibliothèques nécessaires** depuis **TensorFlow** et **Keras** pour construire un modèle de réseau de neurones.
2. **Créer un modèle Sequential** en utilisant la classe `Sequential()`.
Quel est le **but d'un modèle Sequential** dans les réseaux de neurones ?
3. **Ajouter une couche SimpleRNN avec 8 unités.**
Que représente le paramètre **8** dans la couche RNN ?
Quel est le **rôle d'une couche SimpleRNN** ?
4. **Définir la forme d'entrée (input shape) du modèle** comme (5, 3).
Que représente 5 ?
Que représente 3 ?
Comment les **données séquentielles** sont-elles structurées dans ce cas ?
5. **Ajouter une couche Dense avec 1 neurone.**
Quel est le **rôle de la couche Dense** dans ce modèle ?
Pourquoi utiliser **1 neurone de sortie** ?
6. **Utiliser `model.summary()`.**
Quelles informations cette fonction affiche-t-elle ?
Pourquoi est-elle **utile lors de la conception d'un réseau de neurones** ?

Exercice 3: return_sequences parameter

Observer la sortie temporelle du RNN.

1. Créer un modèle **Sequential**.
2. Ajouter une couche **SimpleRNN** avec :
 - **8 unités**
 - `return_sequences=True`
 - `input_shape=(5, 3)`
3. Afficher le résumé du modèle avec `model.summary()`.
4. Expliquer :
 - Quelle est la différence entre `return_sequences=True` et `return_sequences=False` ?
 - Que représente la **sortie temporelle** d'un RNN dans ce contexte ?

Exercice 4 –Introduction aux LSTM et Portes LSTM

1. importer la couche **LSTM** depuis **Keras**.
2. Créer un modèle **Sequential** avec :
 - une couche **LSTM** de **16 unités** et `input_shape=(50, 1)`

- une couche **Dense** avec **1 neurone**.
3. Afficher le résumé du modèle avec `model.summary()`.
 4. Expliquer les **quatre composants principaux** d'un LSTM :
 - **Forget gate (porte d'oubli)** : ... ?
 - **Input gate (porte d'écriture)** : ... ?
 - **Output gate (porte de sortie)** : ... ?
 - **Cell state (état de la mémoire à long terme)** : ... ?
 5. Pourquoi un **LSTM** est-il plus performant qu'un RNN simple pour les séquences longues ?

Exercice 5 –Prédiction de série temporelle

Appliquer un LSTM sur des données séquentielles aléatoires.

1. Générer un vecteur **t** contenant **1000 valeurs** entre 0 et 100 en utilisant **Python** et la bibliothèque **NumPy**.
2. Créer un **signal sinusoïdal** à partir de **t**.
3. Construire les données d'entrée **X** à partir du signal (toutes les valeurs sauf la dernière) et les transformer avec `reshape(-1,1,1)`.
4. Définir la variable cible **y** comme les valeurs du signal **décalées d'un pas**.
5. Créer un modèle **Sequential** avec :
 - une couche **LSTM** de **20 unités** avec `input_shape=(1,1)`
 - une couche **Dense** avec **1 neurone**.
6. Compiler le modèle avec l'optimiseur **Adam Optimizer** et la fonction de perte **Mean Squared Error (MSE)**.
7. Entraîner le modèle pendant **10 epochs** avec `model.fit(X, y, epochs=10)`.

Exercice 6 –LSTM empilé

1. Créer un modèle **Sequential** en utilisant **TensorFlow / Keras**.
2. Ajouter une **première couche LSTM** avec **32 unités** et `return_sequences=True`, avec `input_shape=(10,1)`.
3. Ajouter une **deuxième couche LSTM** avec **16 unités**.
4. Ajouter une **couche Dense** avec **1 neurone** pour la sortie.
5. Expliquer pourquoi on utilise `return_sequences=True` dans la première couche LSTM.
6. Expliquer ce que signifie **empiler (stack) plusieurs couches LSTM** dans un modèle.

Exercice 7 –LSTM (IMDB Sentiment Analysis)

Appliquer un LSTM sur données séquentielles.

Dataset : IMDB Reviews.